Experiment 1: String Constructors

```
public class StringCons{
    public static void main(String[] args) {
        String strLiteral1 = "Java";
        String strLiteral2 = "Java";
        String strEmpty = new String();
        char[] charArray = {'P', 'r', 'o', 'g', 'r', 'a', 'm'};
        String strFromChar = new String(charArray);
        byte[] byteArray = {67, 111, 100, 101};
        String strFromByte = new String(byteArray);
        String strNewObject1 = new String("Java");
        String strNewObject2 = new String("Java");
        System.out.println("=====================================");
        System.out.println("       DISPLAYING CREATED STRINGS        ");
        System.out.println("=====================================");
        System.out.println("1. String Literal: " + strLiteral1);
        System.out.println("2. Empty String via new String(): '" + strEmpty + "' (Length: " +
strEmpty.length() + ")");
        System.out.println("3. From Character Array: " + strFromChar);
        System.out.println("4. From Byte Array: " + strFromByte);
        System.out.println();
        System.out.println("=========================================");
        System.out.println("     LITERAL VS NEW KEYWORD DEMO         ");
        System.out.println("=========================================");
        System.out.print("Comparing two literals (strLiteral1 == strLiteral2): ");
        System.out.println(strLiteral1 == strLiteral2);
        System.out.print("Comparing literal with 'new' object (strLiteral1 == strNewObject1): ");
        System.out.println(strLiteral1 == strNewObject1);
```

```
        System.out.print("Comparing two 'new' objects (strNewObject1 == strNewObject2): ");
        System.out.println(strNewObject1 == strNewObject2);
        System.out.print("Comparing values using .equals() (strLiteral1.equals(strNewObject1)): ");
        System.out.println(strLiteral1.equals(strNewObject1));
    }
}
```

Output:

```
=======================================
      DISPLAYING CREATED STRINGS
=======================================
1. String Literal: Java
2. Empty String via new String(): '' (Length: 0)
3. From Character Array: Program
4. From Byte Array: Code


=======================================
    LITERAL VS NEW KEYWORD DEMO
=======================================
Comparing two literals (strLiteral1 == strLiteral2): true
Comparing literal with 'new' object (strLiteral1 == strNewObject1): false
Comparing two 'new' objects (strNewObject1 == strNewObject2): false
Comparing values using .equals() (strLiteral1.equals(strNewObject1)): true
```

Experiment 2: StringBuffer Class

```
public class StrBuffer{
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Hello");
        System.out.println("Initial String: " + sb);
```

```
        displayStats(sb);
        sb.append(" World");
        System.out.println("\nAfter Append: " + sb);
        displayStats(sb);
        sb.insert(6, "Beautiful ");
        System.out.println("\nAfter Insert: " + sb);
        displayStats(sb);
        sb.replace(6, 15, "Awesome");
        System.out.println("\nAfter Replace: " + sb);
        displayStats(sb);
        sb.delete(5, 13);
        System.out.println("\nAfter Delete: " + sb);
        displayStats(sb);
        sb.reverse();
        System.out.println("\nAfter Reverse: " + sb);
        displayStats(sb);
    }
    public static void displayStats(StringBuffer buffer) {
        System.out.println("-> Length: " + buffer.length() + " | Capacity: " + buffer.capacity());
    }
}
```

Output:

```
Initial String: Hello
-> Length: 5 | Capacity: 21
After Append: Hello World
-> Length: 11 | Capacity: 21
After Insert: Hello Beautiful World
-> Length: 21 | Capacity: 21
```

After Replace: Hello Awesome World

-> Length: 19 | Capacity: 21

After Delete: Hello World

-> Length: 11 | Capacity: 21

After Reverse: dlroW olleH

-> Length: 11 | Capacity: 21

Experiment 3: StringTokenizer Class

```
import java.util.Scanner;
import java.util.StringTokenizer;
public class StringTokenizerDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String sentence = sc.nextLine();
        StringTokenizer st = new StringTokenizer(sentence);
        int count1 = st.countTokens();
        System.out.println("\nTokens:");
        while (st.hasMoreTokens()) {
            System.out.println(st.nextToken());
        }
        System.out.println("Total number of tokens: " + count1);
        System.out.print("\nEnter a delimiter: ");
        String delimiter = sc.nextLine();
        StringTokenizer st2 = new StringTokenizer(sentence, delimiter);
        int count2 = st2.countTokens();
        System.out.println("\nTokens using delimiter '" + delimiter + "':");
        while (st2.hasMoreTokens()) {
```

```
            System.out.println(st2.nextToken());
        }
        System.out.println("Total number of tokens: " + count2);
        sc.close();
    }
}
```

Output:

```
Enter a sentence: Java is easy to learn
Tokens:
Java
is
easy
to
learn
Total number of tokens: 5
Enter a delimiter: a
Tokens using delimiter 'a':
J
v
 is e
sy to le
rn
Total number of tokens: 5
```

Experiment 4: Basic Inheritance

```
class Person {
    String name;
    int age;
```

```
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    void displayPersonDetails() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
class Student extends Person {
    int rollNo;
    String branch;
    Student(String name, int age, int rollNo, String branch) {
        super(name, age);
        this.rollNo = rollNo;
        this.branch = branch;
    }
    void displayStudentDetails() {
        displayPersonDetails();
        System.out.println("Roll Number: " + rollNo);
        System.out.println("Branch: " + branch);
    }
}
public class BasicInheritance {
    public static void main(String[] args) {
        Student s = new Student("Ravi", 19, 101, "CSE-AIML");
        s.displayStudentDetails();
    }
```

```
}
```

Output:

Name: Ravi

Age: 19

Roll Number: 101

Branch: CSE-AIML

Experiment 5: Using `super` Keyword

```
class Employee {
    String name;
    double salary;
    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
    void displayDetails() {
        System.out.println("Employee Name: " + name);
        System.out.println("Salary: " + salary);
    }
}
class Manager extends Employee {
    String department;
    double bonus;
    Manager(String name, double salary, String department, double bonus) {
        super(name, salary);
        this.department = department;
        this.bonus = bonus;
    }
```

```
    void displayManagerDetails() {
        System.out.println("Manager Details:");
        super.displayDetails();
        System.out.println("Employee Name using super: " + super.name);
        System.out.println("Department: " + department);
        System.out.println("Bonus: " + bonus);
    }
}
public class SuperKeywordDemo {
    public static void main(String[] args) {
        Manager m = new Manager("Ravi”,50000, "AI & ML",10000);
        m.displayManagerDetails();
    }
}
```

Output:

Manager Details:

Employee Name: Ravi

Salary: 50000.0

Employee Name using super: Ravi

Department: AI & ML

Bonus: 10000.0